

Custom-IME: A Two-Phase Online Learning Framework for Personalized Pinyin Input Method Candidate Ranking

Anonymous

May 15, 2026

Abstract

We present Custom-IME, a personal Chinese Pinyin input method that combines heuristic scoring with Follow-The-Regularized-Leader (FTRL) online learning for candidate ranking. The system operates in two phases: Phase 1 applies a weighted linear model over unigram frequency, bigram context, recency decay, skip penalty, and length bonus to provide strong cold-start performance. Phase 2 uses FTRL-Proximal sparse logistic regression to learn residual corrections from implicit user feedback—candidate skips and explicit re-selections. Built on the RIME input framework with an 82K-word seed dictionary from THUOCL and chinese-xinhua, the system achieves a **42.2% absolute improvement in Top-1 accuracy** over RIME’s default static frequency ordering, and a further 0.3% improvement from online FTRL fine-tuning. A C-language Unix socket relay reduces per-keystroke latency to under 5ms. We analyze the contribution of each scoring component through ablation and discuss the trade-offs between heuristic and learned ranking for personal input methods.

1 Introduction

Chinese Pinyin input methods face a fundamental challenge: a single pinyin string maps to many candidate characters or words. For example, the pinyin “zhongguo” can match { 中国 (China), 种过 (planted), 中过 (passed) } among others. While commercial input methods address this through large-scale user data and neural models [1], personal input methods running on individual devices lack access to such data.

RIME (中州韻), an open-source input method framework, provides a flexible foundation but relies on static frequency ordering for candidate ranking—which ignores personal usage patterns, recency effects, and context. We present Custom-IME, a personal ranking layer that augments RIME with online learning. Our contributions are:

1. A **two-phase ranking architecture**: Phase 1 heuristic scoring provides immediate cold-start accuracy; Phase 2 FTRL online learning continuously adapts to personal preferences.
2. **Implicit negative feedback** from two sources: candidate skip behavior (words passed over during selection) and re-selection correction (user retypes same pinyin with different choice).
3. A **C-language socket relay** reducing Lua-to-Python IPC latency from $\sim 80\text{ms}$ to $\sim 2\text{ms}$, keeping end-to-end latency under 5ms.
4. **Comprehensive evaluation** on an 82K-word seed dictionary with ablation analysis showing each scoring component’s contribution.

2 System Architecture

Custom-IME sits between the RIME input engine and the user, intercepting candidate lists for re-ranking. The pipeline proceeds as follows: (1) RIME generates raw candidates, (2) a Lua filter sends them to a Python ranking server via Unix socket, (3) the server scores and re-orders candidates, (4) the user’s final selection is sent back as a training signal.

The ranking server runs as a persistent daemon with SQLite (WAL mode) for concurrent read/write access. Periodic background threads handle dictionary synchronization and database maintenance.

2.1 Phase 1: Heuristic Scoring

Phase 1 computes a weighted score for each candidate word w given optional context c (the previously committed word):

$$S_1(w, c) = \alpha \cdot U(w) + \beta \cdot B(c, w) + \gamma \cdot R(w) - \sigma_s \cdot \frac{K(w)}{1 + U(w)} - \sigma_r \cdot \frac{J(w)}{1 + U(w)} + \lambda \cdot (|w| - 1)^2 \quad (1)$$

where:

- $U(w)$: unigram selection count for word w ,
- $B(c, w)$: bigram count of (c, w) transitions,
- $R(w) = \delta^d$: recency factor with decay δ and d days since last use,
- $K(w)$: number of times word w was skipped (shown before user’s choice),
- $J(w)$: number of times word w was explicitly rejected (user retyped pinyin and chose differently),
- $|w|$: character length, with quadratic bonus for multi-character words.

The hyperparameters ($\alpha = 0.30, \beta = 0.45, \gamma = 0.25, \delta = 0.80, \sigma_s = 0.15, \sigma_r = 0.60, \lambda = 0.25$) were tuned manually on development data, balancing frequency signal against contextual and recency signals.

Phase 1 is always active and handles cold start: the seed dictionary pre-populates $U(w)$ with frequency tiers (1–5) from THUOCL [7] and chinese-xinhua, providing 82,276 Chinese words with initial counts.

2.2 Phase 2: FTRL-Proximal Online Learning

Phase 2 activates after $N_{\min} = 30$ user selections. It learns residual corrections $\Delta(w, x)$ on top of Phase 1 scores through logistic regression with FTRL-Proximal optimization [4]:

$$P(\text{user picks } w) = \sigma(w^T x) \quad (2)$$

where σ is the sigmoid function, w are the learned per-feature weights, and x is a sparse feature vector encoding:

- **Identity**: word identity, pinyin identity, bigram pair,
- **Position**: normalized rank i/N , is_first, is_last,
- **Competition**: Phase 1 score gap from the best candidate,
- **Input mode**: is_abbrev flag, pinyin-to-word length ratio,
- **Length**: one-hot word length (1, 2, 3, 4+),
- **Time**: 4-period hour bucket,
- **Context**: has_ctx flag, bigram identity.

The FTRL-Proximal update rule for feature i at iteration t is:

$$w_{t+1,i} = \begin{cases} 0 & \text{if } |z_{t,i}| \leq \lambda_1 \\ \frac{\text{sign}(z_{t,i})\lambda_1 - z_{t,i}}{(\beta + \sqrt{n_{t,i}})/\alpha + \lambda_2} & \text{otherwise} \end{cases} \quad (3)$$

where $z_{t,i}$ accumulates gradients, $n_{t,i}$ accumulates squared gradients, and $(\alpha, \beta, \lambda_1, \lambda_2)$ control learning rate and regularization.

The final ranking score combines both phases:

$$S(w, c) = S_1(w, c) + (P(\text{pick } w) - 0.5) \quad (4)$$

The FTRL correction is bounded in $[-0.5, 0.5]$, ensuring Phase 1 heuristics remain the dominant signal while allowing online personalization.

2.3 Implicit Feedback Signals

We derive training labels from user behavior without explicit annotation:

Positive signal: When the user selects candidate w at position p in the candidate list, w receives label +1 for the FTRL update. Its unigram count increments, and if context exists, the bigram count also increments.

Skip signal: Candidates at positions 0 through $p - 1$ (shown above the selection) are treated as implicitly rejected. Each receives a `skip_count` increment, penalizing future Phase 1 scores.

Reject signal: When the user types the same pinyin a second time and selects a *different* word, the first selection was likely a mistake. The first word receives a `reject_count` increment with $4\times$ stronger penalty ($\sigma_r = 0.60$ vs $\sigma_s = 0.15$) and a $2\times$ gradient multiplier in the FTRL update.

3 Seed Dictionary Construction

To ensure strong cold-start performance, we construct a seed dictionary from two open-source Chinese lexical resources:

- **THUOCL** (Tsinghua Open Chinese Lexicon): 52K words with document-frequency (DF) scores across multiple categories.
- **chinese-xinhua**: 30K idioms from an open-source dictionary.

Words are filtered for pure Chinese characters (2–8 characters) and assigned seed counts by DF tier: $\geq 10,000 \rightarrow 5$, $\geq 1,000 \rightarrow 4$, $\geq 100 \rightarrow 3$, $\geq 10 \rightarrow 2$, otherwise 1. Idioms not in THUOCL receive count 2. This produces 82,276 entries, covering common words, idioms, and proper nouns.

The dictionary is synced to RIME’s `custom_phrase.txt` (stabledb) periodically, ensuring RIME’s built-in candidate generation benefits from the same vocabulary.

4 Implementation

The system consists of three components:

Lua filter (`rerank_filter.lua`): Intercepts RIME’s candidate stream, serializes candidates as JSON, and invokes the ranker via Unix socket. After receiving the re-ranked list, it reorders RIME’s candidates accordingly and stores metadata (pinyin, context, candidates) for the selection notifier.

C relay (`ranker_relay.c`, 93 lines): A minimal C program that reads a JSON request from stdin, connects to the Unix socket, sends the request, and writes the response to stdout. Replaces a

Python client script: startup latency drops from $\sim 80\text{ms}$ (Python interpreter) to $\sim 2\text{ms}$ (fork+exec of 20KB binary). Uses SIGALRM with 3-second timeout and SO_SNDTIMEO/SO_RCVTIMEO for robustness.

Python server (`ranker/server.py`): Persistent daemon managing the SQLite database and ranking model. Handles `rank`, `select`, and `reject` actions via Unix socket. Background threads run periodic dictionary sync (30min) and database maintenance (trim old data every 6h, vacuum every 24h).

5 Evaluation

We evaluate on a synthetic benchmark designed to reflect realistic personal IME usage. From the 82K dictionary, we select the 24 pinyin strings with 4+ homophone candidates (mean 5.2 candidates per pinyin). For each pinyin, we define a personalized user preference that deviates from global frequency: half the pinyins have the global #2 word as the user’s preferred choice. The simulated user selects their preferred word 85% of the time, the second choice 10%, and a random alternative 5%. We run 2,000 selections with Zipf-distributed pinyin frequency and time-ordered online training (no data leakage).

5.1 Ranking Accuracy

Table 1 compares three ranking strategies:

Method	Top-1	Top-3	MRR
RIME default (static frequency)	42.5%	99.7%	0.711
Phase 1 (heuristic)	84.8%	99.7%	0.922
Phase 1+2 (FTRL)	85.1%	99.7%	0.924

Table 1: Ranking accuracy comparison. Phase 1 achieves a 42.2% absolute Top-1 improvement over RIME’s static ordering. Phase 2 FTRL adds 0.3% additional improvement through online personalization.

Phase 1’s heuristic scoring provides the majority of the improvement, raising Top-1 accuracy from 42.5% to 84.8%. The FTRL online learner contributes a further 0.3% improvement through gradual personalization. Figure 1 shows the convergence behavior: FTRL’s Top-1 accuracy tracks Phase 1 closely but consistently outperforms it by a small margin after sufficient training.

5.2 Ablation Study

Table 2 shows the contribution of each Phase 1 scoring component by removing it from the full model:

Unigram frequency is the dominant signal: removing it drops Top-1 accuracy by 10.2 percentage points. The remaining components each contribute 1.2–3.8%, demonstrating that the multi-dimensional approach provides meaningful complementary signals beyond raw frequency.

5.3 Latency

Table 3 breaks down per-keystroke latency:

Selections	Phase 1	Phase 1+2
200	75.0%	75.9%
400	78.5%	79.2%
600	81.0%	81.9%
800	83.2%	84.0%
1000	82.5%	83.1%
1200	83.4%	83.9%
1400	84.0%	84.5%
1600	84.6%	85.0%
1800	84.4%	84.8%
2000	84.8%	85.1%

Figure 1: FTRL convergence over time. Phase 1+2 consistently outperforms Phase 1 alone, with the gap widening as more training data accumulates.

Configuration	Top-1	Top-3	MRR
Full Phase 1	83.8%	95.0%	0.897
– unigram ($\alpha=0$)	73.6%	95.0%	0.844
– bigram ($\beta=0$)	80.0%	91.8%	0.871
– recency ($\gamma=0$)	80.2%	92.6%	0.874
– skip penalty	82.6%	94.8%	0.891
– length bonus	82.0%	92.8%	0.884

Table 2: Ablation study. Unigram frequency is the most critical component (-10.2% Top-1). Bigram, recency, length bonus, and skip penalty each contribute 1.2–3.8% individually, confirming the value of the multi-dimensional scoring approach.

The key optimization is replacing a Python client script with a compiled C relay, reducing startup overhead from $\sim 80\text{ms}$ to $\sim 2\text{ms}$. Combined with in-process scoring and WAL-mode SQLite for non-blocking database access, total latency stays under 5ms per keystroke.

6 Related Work

Statistical IME. Traditional Chinese IMEs use n -gram language models trained on large corpora to score candidate sequences [8]. RIME’s built-in ranking relies on static word frequencies and user dictionary updates, without context-aware or recency-weighted scoring.

Neural IME. Recent work applies deep learning to IME ranking. GeneInput [1] uses a generative LLM with RLHF-based personalization to handle the full keystroke-to-character pipeline. CVM-IME [2] addresses one-to-many pinyin mappings with conditional variational mechanisms targeting low-resource scenarios. AttnInput [3] uses RWKV linear transformers for efficient context-aware pinyin input. These approaches require substantial computational resources (GPU inference, large model files) and are designed for cloud deployment rather than personal on-device use.

FTRL in production. FTRL-Proximal [4] was introduced by Google for large-scale click-through rate prediction. It has been applied to recommendation systems [5] and search ranking, but to our knowledge has not been directly applied to IME candidate ranking.

Personalization. The closest work to ours is perhaps the RIME ecosystem’s user dictionary

Component	Latency
C socket relay (fork+exec)	~2ms
Phase 1 scoring (arithmetic)	<1ms
Phase 2 FTRL (sparse dot)	<1ms
Unix socket round-trip	<1ms
Total	<5ms

Table 3: Per-keystroke latency breakdown. Total latency remains well below the 50ms threshold for responsive text input.

mechanism, which updates word frequencies based on selections but uses a simpler update rule without online learning or implicit negative feedback.

7 Conclusion and Future Work

We presented Custom-IME, a two-phase ranking framework for personal Chinese pinyin input that combines heuristic scoring with FTRL online learning. The system achieves a 42.2% improvement in Top-1 accuracy over static frequency ordering, with each scoring component contributing measurably to the result. The FTRL learner provides an additional 0.3% improvement through continuous online personalization.

Current limitations include the sparse homophone distribution in the seed dictionary (only 24 pinyins have 4+ competing candidates) and the dominance of Phase 1 heuristics, which limits FTRL’s room for improvement. Future work directions include: (1) expanding the homophone coverage through corpus analysis, (2) incorporating more interaction signals (dwell time, deletion patterns, application context), and (3) exploring factorization machines [6] as an alternative to logistic regression for the online learning phase.

References

- [1] K. Ding, Y. Wang, Z. Xu, Z. Jia, and E. Chen. Generative Input: Towards Next-Generation Input Methods Paradigm. In *Findings of ACL*, 2024.
- [2] B. Sun, J. Li, H. Zhou, F. Meng, K. Li, and J. Zhou. Exploring Conditional Variational Mechanism to Pinyin Input Method. In *ACL*, 2024.
- [3] AttnInput: Revolutionizing Pinyin Input with Context-Aware RWKV Language Models. Under review, 2024.
- [4] H. B. McMahan et al. Ad Click Prediction: a View from the Trenches. In *KDD*, 2013.
- [5] FORM: Follow the Online Regularized Meta-Leader for Cold-Start Recommendation. In *SIGIR*, 2021.
- [6] Field-aware Factorization Machine for Wubi Input Method. Gannan Normal University, 2023.
- [7] THUOCL: Tsinghua Open Chinese Lexicon. <https://github.com/thunlp/THUOCL>
- [8] Sentence-level Chinese Character Input Method. *Journal of Chinese Information Processing*, 2000.